receiver. More precisely, by encrypting a message with her own private key a user can prove to another user that she is in fact the source of the message. The receiver can verify the identity of the sender by decrypting the message with the sender's public key. If the operation succeeds, the receiver can be confident that the message was sent by the sender. The process of encrypting a message with a users private key is called *signing* a message.

2.3 Attack Detection

Attack detection assumes that an attacker can obtain access to her desired targets and is successful in violating a given security policy. Mechanisms in this class are based on the optimistic assumption that, most of the time, the information is transferred without interference. When undesired actions occur, attack detection has the task of reporting that something went wrong and to react in an appropriate way. In addition, it is often desirable to identify the exact type of attack. An important facet of attack detection is recovery. Often it is enough to just report that malicious activity has been detected, but some systems require that the effects of an attack be reverted or that an ongoing and discovered attack is stopped. On one hand, attack detection has the advantage that it operates under the worst-case assumption that the attacker gains access to the communication channel and is able to use or modify the resource. On the other hand, detection is not effective in providing confidentiality of information. When the security policy specifies that interception of information has a serious security impact, then attack detection is not an applicable mechanism.

The most important members of the attack detection class are intrusion detection systems. Because this book is focused on intrusion detection and alert correlation, the remaining sections of this chapter are dedicated to a more detailed introduction to intrusion detection.

3. Intrusion Detection

An intrusion is defined as a sequence of related actions performed by a malicious adversary that results in the compromise of a target system. It is assumed that the actions of the intruder violate a given security policy. The existence of a security policy that states which actions are considered malicious and should be prevented is a key requisite for an intrusion detection system. Violations can only be detected when actions can be compared against given rules.

Intrusion detection (ID) is the process of identifying and responding to malicious activities targeted at computing and network resources. This definition introduces the notion of intrusion detection as a process, which involves technology, people, and tools. Intrusion detection is an approach that is complementary with respect to mainstream approaches to security, such as access control and

cryptography. The adoption of intrusion detection systems is motivated by several factors:

1 Surveys have shown that most computer systems are flawed by vulnerabilities, regardless of manufacturer or purpose [Landwehr et al., 1994], that the number of security incidents is continuously increasing [CERT, 2003], and that users and administrators are generally very slow in applying fixes to vulnerable systems [Rescorla, 2003]. As a consequence, many experts believe that computer systems will never be absolutely secure [Bellovin, 2001].

2 Deployed security mechanism, e.g., authentication and access control, may be disabled as a consequence of misconfiguration or malicious actions.

3 Users of the system may abuse their privileges and perform damaging activities.

4 Even if an attack is not successful, in most cases it is useful to be aware of the compromise attempt.

Intrusion detection systems (IDSs) are software applications dedicated to detect intrusions against a target network. IDSs have been designed to address the issues described above. As such, they are not intended to replace traditional security methods, but rather to complete them. According to [Dacier et al., 1999], an intrusion detection system has to fulfill the following requirements.

- *Accuracy* - An IDS must not identify a legitimate action in a system environment as an anomaly or a misuse (a legitimate action which is identified as an intrusion is called a *false positive*).
- *Performance* - The IDS performance must be sufficient enough to carry out real time intrusion detection (real time means that an intrusion has to be detected before significant damage has occurred – according to [Ranum, 2000] this should be under a minute).
- *Completeness* - An IDS should not fail to detect an intrusion (an undetected intrusion is called a *false negative*). One has to admit that it is rather difficult to fulfill this requirement because it is almost impossible to have a global knowledge about past, present, and future attacks.
- *Fault Tolerance* - An IDS must itself be resistant to attacks.
- *Scalability* - An IDS must be able to process the worst-case number of events without dropping information. This point is especially relevant for systems that correlate events from different sources at a small number of dedicated hosts. As networks grow bigger and get faster, such nodes become overwhelmed by the increasing number of events.

3.1 Architecture

There exist many IDSs, based on different conceptual frameworks. It is still possible, however, to recognize a common architecture that underlies all intrusion detection systems. We will present the main components of IDSs and their functionality.

To this end, we use the terminology introduced by the working group on the Common Intrusion Detection Framework (CIDF) [Porras et al., 1998]. CIDF models an IDS as an aggregate of four components with specific roles:

- *Event boxes (E-boxes).* The role of event boxes is to generate events by processing raw audit data produced by the computational environment. A common example of an E-boxes is a program that filters audit data generated by an operating system evaluated at the C2 level of TCSEC [U.S. Department of Defense, 1985]. Another example is a network sniffer that generates events based on the network traffic.

- *Analysis boxes (A-boxes).* The role of an analysis box is to analyze the events provided by other components. The results of the analysis are sent back to the system as additional events, typically representing alarms. Usually A-boxes analyze simple events supplied by E-boxes. Some A-boxes analyze events produced by other A-boxes and operate at a higher level of abstraction.

- *Database boxes (D-boxes).* Database boxes simply store events, guaranteeing persistence and allowing postmortem analysis.

- *Response boxes (R-boxes).* Response boxes consume messages that carry directives about actions to be performed as a reaction to a detected intrusion. Typical actions include killing processes, resetting network connections, and modifying firewall settings.

Figure 2.3 presents an IDS system where two E-boxes monitor the environment and deliver audit events to two A-boxes. These A-boxes analyze the audit data and provide their conclusions to a third A-box that correlates the alerts, stores them in a D-box and controls an R-box. Dashed lines are used to indicate exchange of events, solid lines represent the exchanging of raw audit data.

Note that components are logical entities which produce or consume events. The CIDF model does not mandate what their implementation should be. It only states their roles and interactions. They can be realized as a single process on a single computer or as a collection of cooperating processes spanning multiple computers.

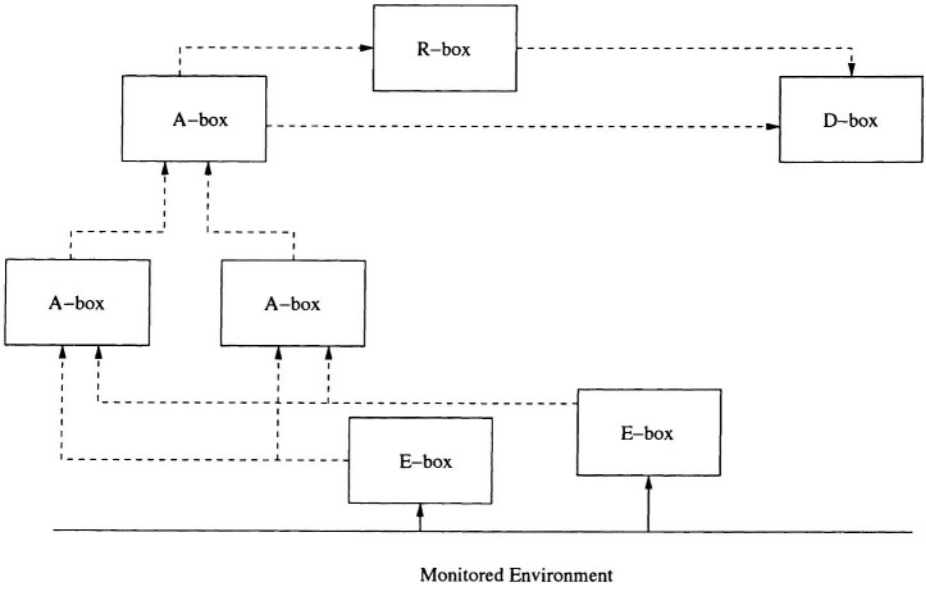

Figure 2.3. CIDF Description of an IDS System

3.2 Taxonomy

Intrusion detection systems may be classified according to different characteristics [Lunt, 1993; Allen et al., 2000; Bace and Mell, 2001]. The following ones seem to be particularly characterizing [Dacier et al., 1999]:

- *Detection method.* It defines the philosophy on which the A-box is built. Two approaches have been proposed. When the IDS defines what is "normal" in the environment and flags as attacks deviations from normality, it is qualified as *anomaly-based.* When the IDS explicitly defines what is "abnormal", using specific knowledge about the attacks in order to detect them, it is called *misuse-based.*

- *Behavior on detection.* It defines a characteristic of the R-box. It is said to be passive if the system just issues an alert when an attack is detected. If more proactive actions are taken (e.g., disconnecting users, shutting down network connections), it is said to be active.

- *Audit source location.* It specifies where the E-box takes audit data from. We distinguish between host-based IDSs, which deal with audit data generated on a single host, e.g., a C2 audit trail; application-based IDSs, which work on audit records produced by a specific application; and network-based IDSs, which monitor network traffic.

- *Usage frequency.* It discriminates between systems that analyze the data in real time and those that are run periodically (offline). It specifies how often the A-box analyzes data collected by other parts of the system.

We will analyze these characteristics more in detail in the following sections.

3.3 Detection Method

Detection can be performed according to two complementary strategies:

1 defining what is the manifestation of an attack and searching for an occurrence of the attack (misuse-based) or

2 defining what is the normal behavior on the system and searching for activities that deviate from it (anomaly-based).

Misuse-based Systems

Misuse-based systems are equipped with a database of information (the knowledge base) that contains a number of attack models (sometimes called "signatures"). The audit data collected by the IDS is compared with the content of the database and, if a match is found, an alert is generated. Events that do not match any of the attack models are considered part of legitimate activities.

The main advantage of misuse-based systems is that they usually produce very few false positives: attack description languages usually allow for modeling of attacks at such fine level of detail that only a few legitimate activities match an entry in the knowledge base.

However, this approach has drawbacks as well. First of all, populating the knowledge base is a difficult, resource intensive task. Furthermore, misuse-based systems cannot detect previously unknown attacks, or, at most, they can detect only new variations of previously modeled attacks. Therefore, it is essential to keep the knowledge base up-to-date when new vulnerabilities and attack techniques are discovered.

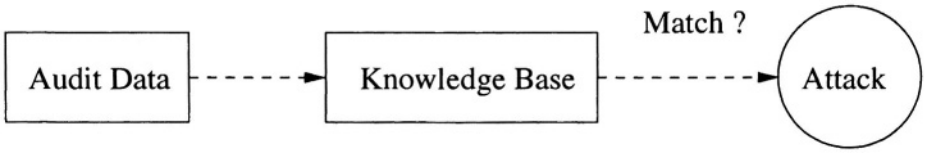

Figure 2.4. Block Diagram of a Typical Knowledge-Based IDS

A number of different techniques have been proposed for performing misuse-based intrusion detection [Ilgun et al., 1995; Lindqvist and Porras, 1999; Kumar and Spafford, 1994; Roesch, 1999; Neumann and Porras, 1999]. Before briefly

discussing the most common misuse-based techniques, it is worth discussing how these systems can be differentiated on the basis of state maintenance (and the importance of this characteristic on the overall intrusion detection system).

A *stateless* intrusion detection system treats each event independently of others. When the processing of the current event is completed, every information regarding this event is discarded.

This approach simplifies the design of the system, specifically of the A-box, because it does not need to allocate and maintain memory to keep information about past activity. Stateless systems usually have very good performance in terms of speed of processing because the analysis step is reduced to simple lookups in the knowledge base and matches against the current event, with no additional operations needed.

However, stateless systems have important limitations. In the first place, they are not complete, in the sense that there exist classes of attacks that cannot be detected. For example, multistep attacks are malicious activities that require a series of steps to be completed. Actions involved in each step are not *per se* intrusive and become malicious only when executed in the correct and complete sequence. Because the system has no memory of past events, it lacks the possibility to keep track of steps. Note that it would be possible to model the entire attack on the basis of the last step, thus considering the action involved in this step as malicious, and inserting it in the attack knowledge base. However, because this action alone is not intrusive, it is likely to be performed also as part of legitimate activities. Thus, this approach would possibly generate an unacceptable number of false positives, thwarting the main advantage of misuse-based intrusion detection.

Furthermore, stateless systems are subject to a class of attacks whose aim is to trigger the generation of a flood of alerts. A number of tools have been proposed that analyze the database of attacks and automatically generate events or series of events that conform to the attack descriptions. The event stream synthesized in this manner forces intrusion detection systems to generate a large number of detection alerts. The resulting "alert storm" has been used to desensitize intrusion detection system administrators and hide attacks in the event stream [Patton et al., 2001; Mutz et al., 2003; Snot, 2004; Stick, 2004].

Stateful intrusion detection systems maintain information about past events. As a consequence, the effect of a certain event on the system is not independent of its position in the event stream. While this approach adds an additional level of complexity to the design and implementation of A-boxes, it provides significant advantages. In particular, stateful tools are able to model and detect attacks that involve many steps. Furthermore, they are less prone to the alert storm attacks described in the previous paragraph, because it is more difficult to develop a program that is able to exactly reproduce the steps of a modeled attack. However, these systems are all vulnerable to state-based attacks, where an

attacker forces the IDS to maintain large amount of information about ongoing attacks, effectively affecting the overall performance of the system.

In signature-based systems, abstractions ("signatures") of attacks are stored in the knowledge base. Signatures are used to summarize in a compact format all the information necessary to describe attacks. For example, a network intrusion detection system may store in the knowledge base the content of network packets involved in known attacks. Usually, signatures are stored in a format that allows straightforward comparison with information found in the event stream. During operation, events are checked against entries in the signature file: if a match is found, the system raises an alarm.

Signature based systems have gained popularity because they are easy to develop, give accurate feedback on alarms, and are usually requires little computational resources.

However, they also have disadvantages:

- The description of an attack is usually a very low level description and thus, difficult to determine or interpret.
- Every attack or variation of an attack requires a new signature to be added to the knowledge base, thus its size can become very large.
- The more specific a signature is, the less false positives are generated. But also, the more specific a signature is, the easier it is for an attacker to create a slightly different version of an attack that do not match the signature and thus goes unnoticed. Such an attack is said to be *polymorphic.*

Example of signature-based systems include Snort [Roesch, 1999], EMERALD [Neumann and Porras, 1999] and many commercial products.

One technique that can be used to describe complex attack signatures are state transitions [Ilgun et al., 1995]. State transitions model an activity on the system as a series of state transitions, where a *state* is a snapshot of the system representing the value of all the memory locations of the system. Malicious activities move the state of the system from an initial safe state to a final compromised state, possibly passing through a number of intermediate states. This technique requires the analyst to identify those transitions that are critical in leading the system into a compromised state. The IDS will then search for such transitions. Of course, state transition systems are stateful tools.

State transition models are appealing because they allow high-level, even graphical, description of attacks. They have very high expressive power, meaning that arbitrarily complicated attacks can be modeled and detected. In particular, multistep attacks fit very well the state transition modeling technique. It also provides very detailed feedback on the generated alerts, because the entire sequence of actions that caused the alarm to be triggered can be easily provided. Furthermore, it allows to deploy a response before an attack reaches its final

step, thus allowing to effectively prevent the attack rather than only detecting it.

The main disadvantage of the state transition technique is that its computational requirements can be high if the system has to keep track of many concurrent attacks.

Anomaly-based Systems

Anomaly-based systems are based on the assumption that all anomalous activities are malicious[1]. Thus, they first build a model of the normal behavior of the system (i.e., profile) and then look for anomalous activities, i.e., activities that do not conform to the established model. Anything that does not correspond to the system profile is flagged as intrusive [Helman and Liepins, 1993]. Many systems have been built following this approach, e.g., [Hofmeyr et al., 1998; Ghosh et al., 1998; Tan and Maxion, 2002; Ko et al., 1997].

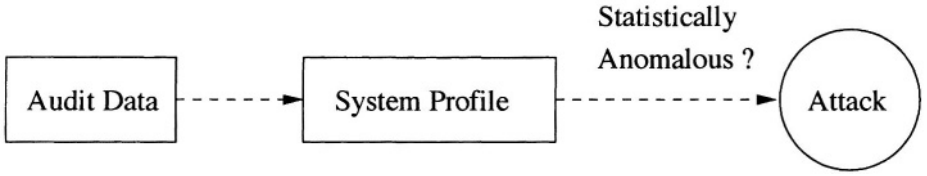

Figure 2.5. Block Diagram of a Typical Behavior-Based IDS

The main advantage of systems based on this approach is that, in theory, they are able to detect previously unknown attacks. On the other hand, anomaly-based systems are prone to the generation of many false positives, that is, their accuracy is typically low.

One class of anomaly detection systems use machine learning techniques to determine the parameters of normal system behavior (i.e., system profiles). These systems may suffer from specific problems:

- The amount of time required to teach to the system the normal behavior might be long.
- The behavior of the monitored environment might change during a period of time, requiring the system to be retrained.

[1] Note that "All malicious activities are anomalous" is different from "All anomalous activities are malicious". The former implies that there do not exist malicious activities that are not anomalous and thus assumes that systems that detect every anomaly also detect every attack. On the other hand, the latter leaves open the possibility that there exist attacks that do not appear as anomalous activity. Thus, the "all malicious is anomalous" assumption postulates that anomaly-based systems can detect every possible attacks. The "all anomalous is malicious" asserts that attacks might go undetected by an anomaly-based IDS.

- If the training set contains attacks, the system will consider malicious behavior normal.
- It may be possible to perform an attack within the boundaries of "normality".

Statistical analysis has shown that, under reasonable assumptions, the probability $P(Intrusion|Alarm)$, i.e., the probability that an alarm really indicates an intrusion, is dominated by the false alarm rate rather than the true positive detection rate [Axelsson, 1999]. This is a consequence of the *base rate fallacy:* typical traffic shows a huge number of benign activities and only a few malicious events. The problem is that most systems today have poor performance in terms of suppression of false alarms.

A number of studies have indicated that most network systems are intrinsically characterized by a high rate of change and diversity [Floyd and Paxson, 2001]. Moreover, ambiguities in underlying protocols and differences in application programs exist [Bellovin, 1993]. Thus, detecting anomalous behavior in certain environments is very difficult, because anomalies are characteristic of the environment itself.

Finally, some new attack strategies have emerged that mimic legitimate activities and thus go undetected [Wagner and Soto, 2002; Tan et al., 2002]. While these studies were applied to a specific anomaly-based IDS, they are based on general concepts that seem to be applicable to whole classes of anomaly-based IDSs.

3.4 Type of Response

Most intrusion detection tools are passive: when an attack is detected, an alert is generated without trying to oppose the attack any further. This requires the security officer to manually inspect all alerts and take appropriate actions. Therefore, there can be a significant delay in the process of dealing with an intrusion.

A number of IDSs have proactive capabilities, e.g., they can change the security posture of a protected network to react to the detected attack. For example, such systems may modify file permissions, add firewall rules, kill processes, or shutdown network connections. It is important to note, however, that automatic countermeasures could, in certain cases, be leveraged by the attacker to damage the system itself or to cause a denial of service.

3.5 Audit Source Location

Intrusion Detection Systems can be characterized according to the source of the events they analyze. Typical system classes include host-based IDSs, application-based IDSs, network-based IDSs, as well as correlation systems. These are discussed in detail in the next paragraphs.

Host-based IDSs

Host-based IDSs (HIDS) detect attacks against a specific host by analyzing audit data produced by the host operating systems. Audit sources include:

- System information. Operating systems make available to processes in user space information about their internal working and security-relevant events. There exist programs that collect and show this information, e.g., *ps, vmstat, top, netstat.* The information provided is usually very complete and reliable because it is retrieved directly from the kernel. Unfortunately, few operating systems provide mechanism to systematically and continuously collect this information.

- Syslog facility [Lonvick, 2001]. Syslog is an audit facility provided by many UNIX-like operating systems. It allows programmers to specify a text message describing an event to be logged. Additional information, like the time when the event happened and the host where the program is running, is automatically added. Because of their simplicity, syslog events are used extensively by applications. However, applications usually log information valuable for debugging purposes that is not necessarily tailored to the needs of intrusion detection. Furthermore, a specific audit format is not imposed by the facility but changes according to the program that uses it. Thus, it may be difficult to extract audit data from logs and perform sophisticated analysis. Finally, the logged information can be easily polluted by messages crafted by an attacker to cover her tracks.

- C2 audit trail. Some operating systems comply with the C2 level of the TCSEC standard and thus monitor the execution of system calls. The data obtained is accurate because it comes directly from within the kernel.

Application-based IDS

Application-based IDSs detect attacks against a specific application. The audit information necessary to perform intrusion detection is usually obtained either using the already described syslog facility or instrumenting the application with specific audit mechanisms. The following discussion will focus on the latter approach.

Adding audit mechanisms to an existing application requires the modification of the application so that it produces audit information in response to security-relevant events. This can be accomplished in different ways:

- Directly modifying the application source code to include auditing code. This approach requires that the application code be available and modifiable.

- Interposing code responsible to extract audit information in correspondence of interfaces used by the application, e.g., the system call or the standard C

library interface. One disadvantage of this approach is that applications other than the one to be monitored could be affected, e.g., in terms of performance loss, by the use of the interposition mechanism.

- Using extension hooks provided by the application itself to implement the audit collection functionality. This approach guarantees great flexibility, but not all the applications offer extension hooks.

A description of a tool performing intrusion detection at the application level can be found in [Almgren and Lindqvist, 2001]. This system leverages the extension mechanism of the Apache web server to implement an audit data source. The collected audit information is used to monitor the behavior of the web server.

Network-based IDSs

Network-based IDSs (NIDSs) detect attacks by analyzing the network traffic exchanged on a network link. Therefore, in network-based IDSs, the E-boxes are network sniffers.

The analysis of the content of network packets can be performed at different levels of sophistication, e.g., performing simple pattern matching on the header and/or the payload of a packet, or exploiting knowledge about the protocol followed by the communication. Higher-level analysis supports more sophisticated analysis of the data, but it is usually slower and requires more resources.

Network-based IDSs are very appealing because they are easy to deploy, and have a minimal or no impact on the monitored hosts. On the other hands, changes in network technology could impair the usefulness of NIDSs:

- High-speed networks might increase the network throughput beyond the capabilities of sniffers.

- Switched networks make it more difficult to choose the location where the NIDSs should be placed.

- The adoption of encryption of the communications reduces or completely prevents NIDSs from accessing the contents of network connections.

Furthermore, studies have shown that, if particular care is not taken, NIDSs are vulnerable to a class of attacks (called insertion and evasion) that take advantage of the physical and logical separation of the NIDSs from their monitored hosts to undermine their detection capability. [Ptacek and Newsham, 1998; Malan et al., 2000; Handley et al., 2001].

3.6 Usage Frequency

Dynamic intrusion detection tools analyze in real time the activity of the system to be protected. Audit data is examined as soon as it is produced. The advantage of this approach is that system activities can be analyzed timely and thus, a proper response can be issued when an attack is detected. However, real-time collection and analysis of audit data may introduce a significant overhead.

Static tools are run offline at specific intervals. They analyze a snapshot of the system state and produce an evaluation of the security of that state. They do not provide any security in between two consecutive runs, and therefore, in case of a successful attack, they can be used only for postmortem analysis. However, by running only occasionally, they may perform a more thorough analysis without having an unacceptable impact on the performance of the monitored system.

3.7 IDS Cooperation and Alert Correlation

A recent trend in intrusion detection is to use, at the same time, different intrusion detection systems and correlate the analysis results and corresponding alerts. This approach aims at achieving higher-level descriptions of attacks or a more condensed view of the security issues highlighted during the analysis.

A number of requirements and possible application scenarios for IDS interoperation have been described [Kahn et al., 1998]. For example, two IDSs might cooperate in the following ways:

- Analyzing each other's generated alerts. This would be especially useful if they use different detection techniques, e.g., one IDS is anomaly-based and the other one is misuse-based.
- Complementing each other's coverage, e.g., two IDSs may be both host-based but located on two different hosts involved in an attack. These IDSs would produce an aggregate report on the malicious activity.
- Reinforcing each other's alerts to keep the number of false positives low.

Alert correlation is a multistep process that receives alerts from different intrusion detection systems as input and produces a high-level description of the malicious activity on the network. The core of this book describes the different phases of the correlation process and the challenges associated with each phase. We emphasize open problems and discuss current approaches to solve some of the issues. After a detailed discussion of the correlation phases, we present an approach to perform distributed correlation in Chapter 7 that addresses scalability requirements for correlation in very large network installations.